Catching SMS Spam with Naive Bayes

CS 5830 · Project 5

Abinash Pant

October 16, 2026

Abstract

Bernoulli Naive Bayes on word presence caught 87.5% of SMS spam in 1,032 unseen messages without flagging a single legitimate one (precision 1.00, F1 0.933, 95% CI 0.896–0.964). I cleaned the UCI SMS Spam Collection from 5,574 to 5,159 unique messages, removing duplicates and a ham-only anonymisation artifact that would otherwise have become the strongest "ham" word. I then compared multinomial, Bernoulli and Gaussian Naive Bayes, a hybrid that adds multinomial and Gaussian log-likelihoods, and logistic regression, all tuned by 5-fold cross-validation on the training set. The discrete event models fit word counts; Gaussian Naive Bayes fits continuous message statistics but fails on word counts (recall 0.68); and the hybrid double-counts correlated length features, adding false alarms. Phone numbers and prize words give spam away, and the spam that slips through reads like ordinary chat.

1. Introduction

A Naive Bayes filter trained on 4,127 text messages caught 87.5% of the spam in 1,032 unseen messages without flagging a single legitimate one. This report explains how, and what the choice between multinomial and Gaussian Naive Bayes changes.

Spam texts push premium-rate numbers, fake prizes and scam links. A filter has two jobs that pull against each other: catch the spam, and never hide a real message. A missed spam is an annoyance, but a lost message from a friend is worse, so false positives cost more than false negatives.

Naive Bayes is a natural first tool. It applies Bayes' rule word by word, trains in milliseconds, learns from few examples (Ng and Jordan, 2002) and shows exactly which words drove each decision. It has been a standard baseline for text and spam filtering since the late 1990s (Sahami et al., 1998; McCallum and Nigam, 1998). It also comes in several *event models*, each assuming a different shape for the features, and this project tests which shape fits which features.

Research questions

1. **RQ1.** How well does Naive Bayes separate spam from ham on messages it has never seen?
2. **RQ2.** Which event model fits which features (multinomial on word counts, or Gaussian on continuous message statistics), and does combining them help?
3. **RQ3.** What gives spam away, and which errors remain?

The project combines the two approaches the assignment describes. The raw data needed real cleaning before any modelling (Section 2), and the Naive Bayes models are paired with other analyses: a logistic-regression baseline, a cost-based decision threshold, an error analysis and a learning curve (Section 4).

All code is in one Jupyter notebook, `sms_spam_naive_bayes.ipynb`. It runs top to bottom in about 30 seconds and produces every number, table and figure in this report; an 8-slide deck summarises the results.

2. Dataset

The UCI SMS Spam Collection (Almeida, Gómez Hidalgo and Yamakami, 2011) holds 5,574 English text messages labelled *ham* (legitimate) or *spam*. After cleaning, 5,159 unique messages remain, 641 of them spam (12.4%). The collection merges four public sources:

Source	Ham	Spam
Grumbletext, a UK forum where people post spam texts they received		425
NUS SMS Corpus, mostly students at the National University of Singapore	3,375	
Caroline Tagg's PhD thesis corpus	450	
SMS Spam Corpus v0.1 Big	1,002	322
Total (raw file)	**4,827**	**747**

2.1 Cleaning and preparation

Each step fixes a specific problem found in the raw file:

1. **Broken characters.** 601 messages contained HTML entities (`<`, `&`) or Windows-1252 punctuation decoded as control codes (`\x92` instead of an apostrophe). They were unescaped and mapped to plain ASCII.
2. **Duplicates.** The raw file has 403 exact duplicates. After the character fix, 415 copies (309 ham, 106 spam) were removed, because a message in both the training and test sets measures memory, not generalisation. No text carried both labels.
3. **A ham-only artifact.** The NUS corpus hid numbers, times and links behind placeholders: <#> appears 254 times, <DECIMAL> 18 times and <TIME> 6 times, all in ham. Left alone, <#> would be the strongest ham word in the vocabulary, making a message 477 times more likely to be ham per occurrence. The placeholders were mapped to the same tokens as real numbers and links.
4. **Normalisation.** Text was lower-cased, and links, e-mail addresses, money amounts, phone numbers (UK numbers and 5-digit short codes) and numbers of three or more digits were replaced with category tokens. One- and two-digit numbers stay as words, since in SMS slang "2" and "4" often mean *to* and *for*. Cross-validated F1 was unchanged (0.955 vs 0.956 on raw text), but the vocabulary shrank by 8% and the model now recognises phone numbers it never saw.
5. **Features.** Words form a 7,000-word bag-of-words vocabulary, with 14.4 distinct words per message on average. Eight numeric statistics are computed per message: characters, words, mean word length, digits, uppercase share, exclamation marks, currency symbols and links.
6. **Split.** A stratified 80/20 split gives 4,127 training messages (513 spam) and 1,032 test messages (128 spam). The test set was not used until the final evaluation.

2.2 What separates the classes

Spam is long and full of numbers: it is written to fill one 160-character message with a prize, a price and a number to call.

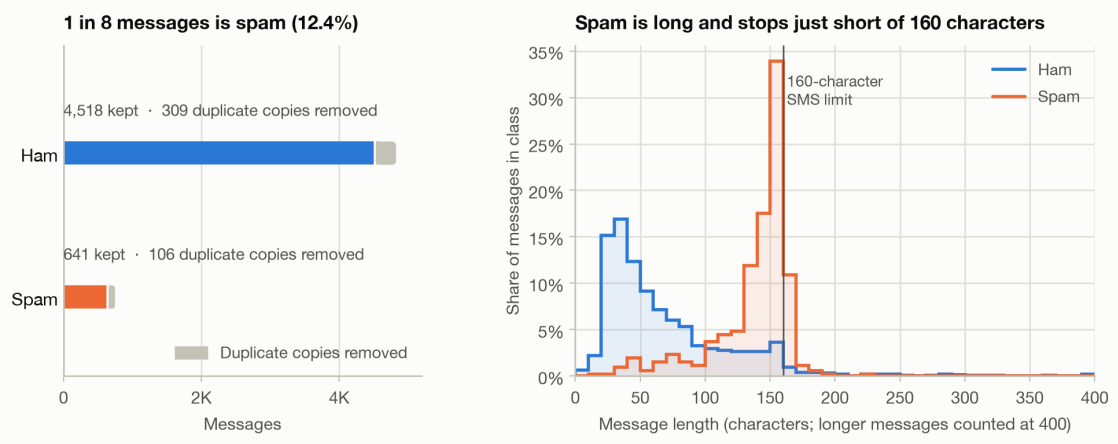


Figure 1. Left: after removing duplicate copies, one message in eight is spam. Right: 51% of spam is 140–159 characters long, just under the 160-character SMS limit, while most ham is under 60 characters (training set).

Statistic (training set)	Ham median	Spam median	Spam above the ham median
Characters	52	148	96%
Words	11	25	95%
Mean word length	4.0	4.9	90%
Digits	0	16	95%
Uppercase share	4.5%	14.9%	89%
Exclamation marks	0	0	48%
Currency symbols	0	0	35%
Links	0	0	14%

Most statistics are skewed or piled up at zero (three have a median of 0 in both classes), which matters for the Gaussian model in Section 3.

3. Analysis Technique

I used both multinomial and Gaussian Naive Bayes, each on the features whose shape it assumes, plus the Bernoulli variant and a hybrid that combines the two.

3.1 Naive Bayes

For a message x and a class $y \in$ {ham,spam}, Bayes' rule gives

$$P(y \mid x) = \frac{P(y)\,P(x \mid y)}{P(x)} \;\propto\; P(y)\,P(x \mid y).$$

The *naive* step assumes the features $x_1, \ldots, x_d$ are independent once the class is known, so $P(x \mid y) = \prod_j P\,(x_j \mid y)$. A product of thousands of small probabilities underflows to zero in floating point, so every model works in logs and predicts the class with the larger score

$$\log P(y) + \sum_j \log P(x_j \mid y).$$

The event models differ only in the shape assumed for $P(x_j \mid y)$:

Model	Feature	Likelihood $P(x \mid y)$	Used here for
Multinomial	word counts x_w	$\prod_w \theta_{yw}^{x_w}$	message words
Bernoulli	word presence $b_w \in \{0,1\}$	$\prod_w \theta_{yw}^{b_w} (1-\theta_{yw})^{1-b_w}$	message words
Gaussian	continuous values x_j	$\prod_j \mathcal{N}(x_j;\, \mu_{yj}, \sigma_{yj}^2)$	eight message statistics

For the word models the probabilities are smoothed, $\theta_{yw} = (N_{yw} + \alpha)/(N_y + \alpha V)$, where N_{yw} counts word w in the training messages of class y, N_y is that class's total word count, V the vocabulary size and α a pseudo-count ($\alpha = 1$ is Laplace smoothing). Without smoothing, one word never seen in ham would force $P(\text{ham} \mid x) = 0$ whatever else the message says. Gaussian Naive Bayes adds a small fraction of the largest feature variance to every variance (variance smoothing) for the same reason.

3.2 Multinomial or Gaussian? Both, each where it fits

- **Words: multinomial.** A message is a bag of word tokens, and the multinomial model is exactly the story "each word is drawn from a class-specific word distribution". Word counts are discrete, non-negative and almost always zero: a message uses about 14 of the 7,000 vocabulary words.
- **Words: Bernoulli, as a check.** SMS messages are so short that a word rarely appears twice, so counts are nearly binary. The Bernoulli model asks only which words are present, and it also scores the absence of typical spam words. McCallum and Nigam (1998) found the Bernoulli model competitive at small vocabularies and the multinomial better at large ones, and Metsis, Androutsopoulos and Paliouras (2006) compared five Naive Bayes variants for e-mail spam.
- **Message statistics: Gaussian.** Length, digits, uppercase share and the rest are measurements, so each gets a class-specific normal distribution. They are right-skewed (Figure 2), so $\log(1+x)$ is applied first.
- **Words: Gaussian, on purpose.** Gaussian Naive Bayes is also fitted to the counts of the 2,000 most frequent words, to show what happens when the event model does not match the data.
- **Hybrid: both at once.** Sahami et al. (1998) found that domain-specific features improve a text-only Naive Bayes spam filter. Under the naive assumption, words and statistics are independent given the class, so their log-likelihoods add:

$$\log P(y \mid x) = \log P(y) + \log P_{\text{multinomial}}(\text{words} \mid y) + \log P_{\text{Gaussian}}(\text{statistics} \mid y) + \text{const.}$$

The notebook implements the hybrid as a scikit-learn-compatible class that adds the two models' joint log-likelihoods and subtracts one copy of the prior, which both contain.

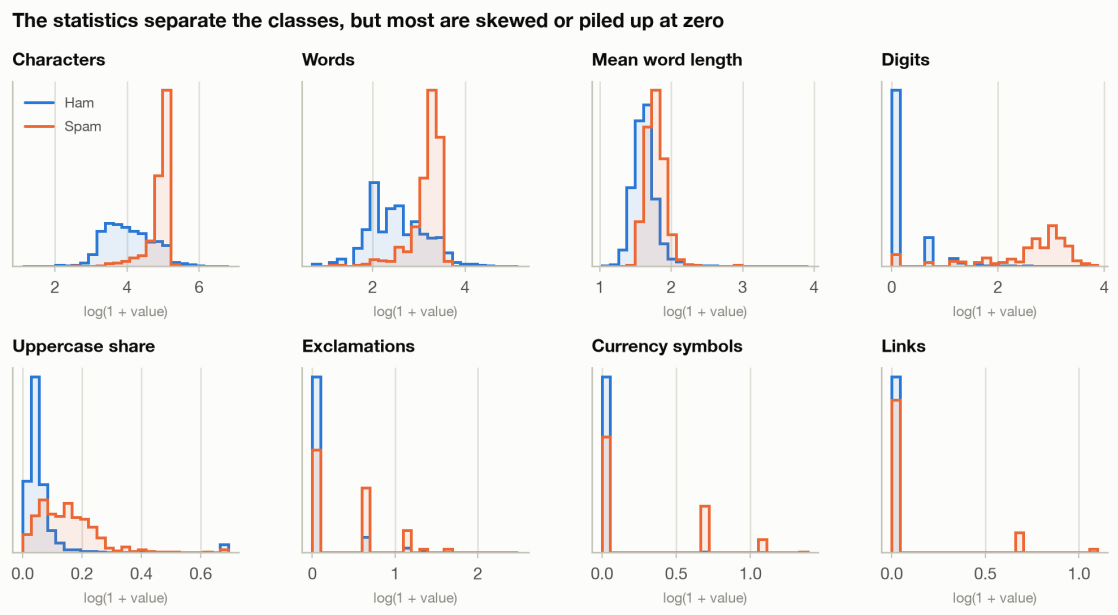


Figure 2. The eight message statistics by class on a log(1 + x) scale (training set). Length becomes roughly bell-shaped after the transform; counts such as links and currency symbols stay piled up at zero.

3.3 Implementation

The notebook uses scikit-learn 1.9 (Pedregosa et al., 2011) for the models and cross-validation. To show that the method is not a black box, it also implements multinomial Naive Bayes from scratch in NumPy, in about ten lines: smoothed log-probabilities, a log prior, and normalisation with log-sum-exp. On the 4,127 training messages the from-scratch and scikit-learn log-probabilities agree to within 1.4×10^{-14}, and every prediction is identical.

3.4 Evaluation protocol

1. **Split once:** 80% training, 20% test, stratified by class (random seed 5830).
2. **Tune on the training set only:** grid search with 5-fold stratified cross-validation, scored by F1 on the spam class. Grids: $\alpha \in \{0.01, 0.03, 0.1, 0.3, 1\}$ and unigrams or unigrams + bigrams for the word models; variance smoothing $\in \{10^{-9}, 10^{-6}, 10^{-3}, 10^{-1}\}$ for Gaussian Naive Bayes; regularisation $C \in \{0.1, 1, 10, 100\}$ for logistic regression.
3. **Select** the final filter as the Naive Bayes model with the best cross-validated F1.
4. **Test once:** every tuned model classifies the test set; we report precision, recall, F1 with a 95% interval from 1,000 bootstrap resamples, ROC AUC and average precision (PR AUC).
5. **Compare** against a majority-class baseline (always “ham”) and L2-regularised logistic regression on the same word counts.

F1 is the selection metric because accuracy rewards the baseline: predicting “ham” for every message is 87.6% accurate and catches nothing.

4. Results

4.1 Model selection

Bernoulli Naive Bayes had the best cross-validated F1 and became the final filter.

Model	Best settings	CV F1 (mean ± sd)
Bernoulli NB · word presence	unigrams, α = 0.1	0.967 ± 0.003
Hybrid NB · words + statistics	unigrams + bigrams, α = 0.1, variance smoothing 10^{-9}	0.963 ± 0.003
Multinomial NB · word counts	unigrams, α = 0.1	0.955 ± 0.006
Logistic regression · word counts	unigrams, C = 100	0.951 ± 0.005
Gaussian NB · 8 message statistics	variance smoothing 10^{-9}	0.886 ± 0.024
Gaussian NB · word counts	variance smoothing 0.1	0.859 ± 0.025
Majority class (always ham)	none	0.000

4.2 Test-set performance

On the 1,032 test messages (904 ham, 128 spam), the final Bernoulli filter flagged no legitimate message and caught 112 of 128 spam.

Model	Accuracy	Precision	Recall	F1 (95% CI)	ROC AUC	PR AUC
Bernoulli NB (final)	0.984	1.000	0.875	0.933 (0.896–0.964)	0.982	0.962
Multinomial NB	0.984	0.975	0.898	0.935 (0.900–0.964)	0.955	0.932
Hybrid NB	0.980	0.935	0.898	0.916 (0.877–0.949)	0.977	0.939
Logistic regression	0.980	0.973	0.859	0.913 (0.870–0.949)	0.962	0.926
Gaussian NB · statistics	0.953	0.794	0.844	0.818 (0.769–0.865)	0.967	0.877
Gaussian NB · word counts	0.956	0.956	0.680	0.795 (0.731–0.852)	0.951	0.869
Majority class	0.876	0.000	0.000	0.000	0.500	0.124

Multinomial and Bernoulli Naive Bayes are statistically tied: their F1 intervals overlap almost completely. Bernoulli's advantages are practical: zero false alarms and the best ranking of messages (ROC AUC 0.982, PR AUC 0.962). Both beat logistic regression, which flagged 3 ham messages and missed 18 spam.

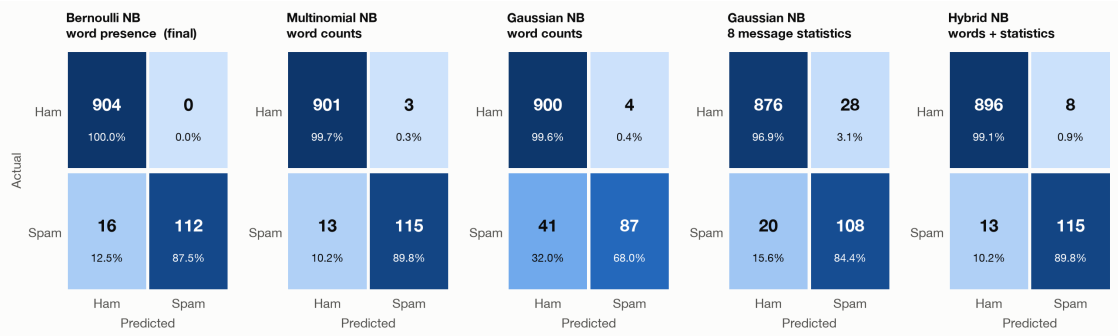


Figure 3. Test-set confusion matrices for the five Naive Bayes models. Rows are the true class and columns the prediction; shading shows the share of each true class.

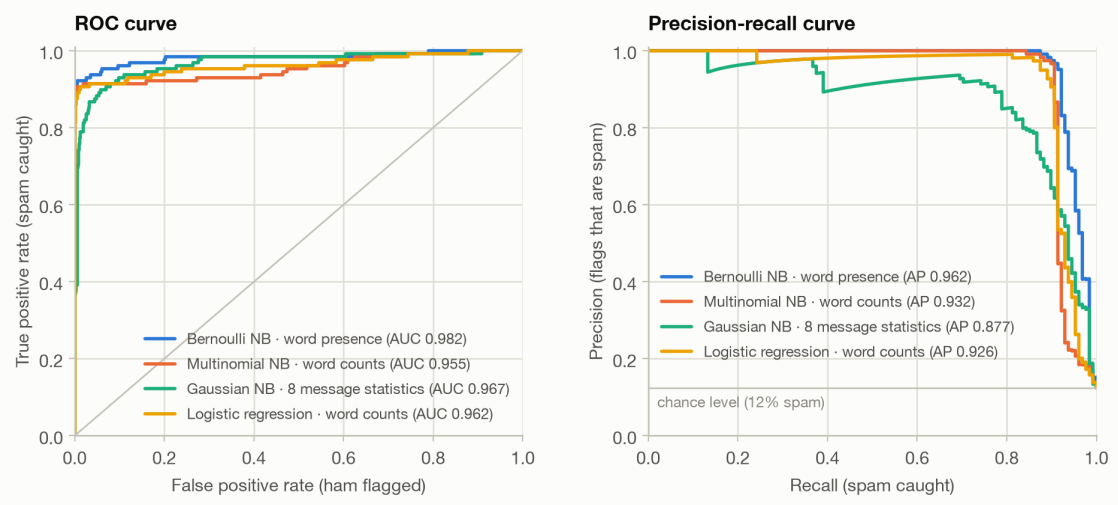


Figure 4. ROC and precision-recall curves on the test set. Multinomial Naive Bayes has the lowest ROC AUC of the four because it ranks 6 spam messages below the median ham message, one of them at P(spam) ≈ 6 × 10^{-18}.

4.3 Multinomial versus Gaussian

Matching the event model to the feature type mattered more than any hyperparameter:

- **Gaussian on word counts fails on recall.** It caught only 87 of 128 spam (recall 0.68). A normal curve fitted to counts that are almost always 0 puts probability on impossible negative and fractional counts and is pulled around by rare large counts.
- **Gaussian on statistics works, within limits.** With only 16 means and 16 variances it reaches F1 0.818, but it flags 28 ham messages: long, number-heavy legitimate messages look like spam on length and digits alone.
- **Combining both did not help.** The hybrid catches as much spam as multinomial Naive Bayes alone (115 of 128) but flags 8 ham messages instead of 3. Characters and words correlate at 0.97 within ham, so the Gaussian part counts message length twice, and the words already carry most of that evidence. This is the independence assumption failing in practice.

4.4 What gives spam away

Naive Bayes is transparent: a word's evidence is the ratio $\theta_{\text{spam},w}/\theta_{\text{ham},w}$, how many times more often it occurs in spam than in ham (Figure 5). Spam evidence is contact and reward words (a phone number, "claim", "prize", "guaranteed", "awarded"), premium-rate jargon ("ppm" for pence per minute, "cs" for customer service, "tone" for ringtone) and age gates ("18", "16"). Ham evidence is everyday chat ("he", "she", "lol", "later", "happy") and Singlish particles ("lor", "da", "ü" for *you*), a reminder that most ham here comes from Singaporean students. The final Bernoulli model ranks these words almost identically (Spearman ρ = 0.998 between the two models' word evidence). Words never seen in ham get ratios set by the smoothing α, so "×19,454" means "very large" rather than an exact multiplier.

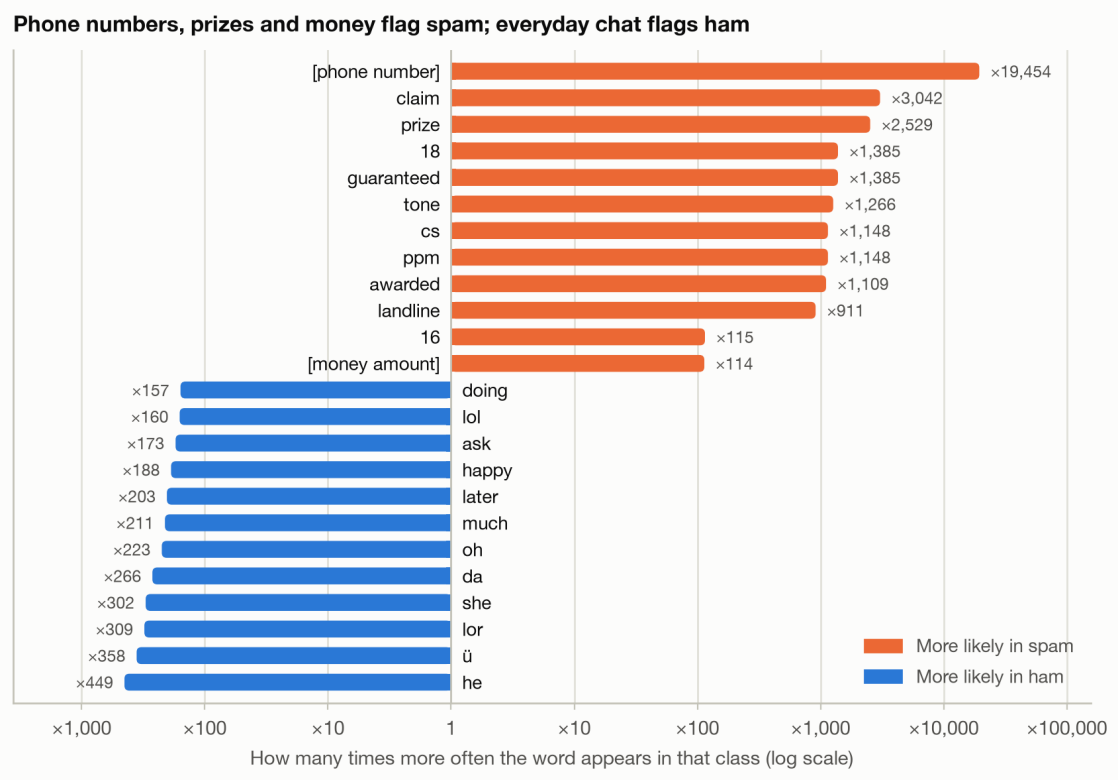


Figure 5. The 12 strongest spam and ham words (multinomial Naive Bayes, unigrams, words seen at least 20 times in training).

4.5 Choosing the decision threshold

A lost real message costs more than a missed spam. Assuming a false positive costs 10 times a false negative, the threshold that minimises that cost on out-of-fold predictions for the training set is P(spam) > 0.993 instead of 0.5. On the training folds it cut false alarms from 5 to 3 at the price of 12 more missed spam. On the test set there were no false alarms to cut, so it only let 3 more spam through (19 vs 16). The default threshold is kept: Bernoulli Naive Bayes is already conservative at 0.5 (Figure 6).

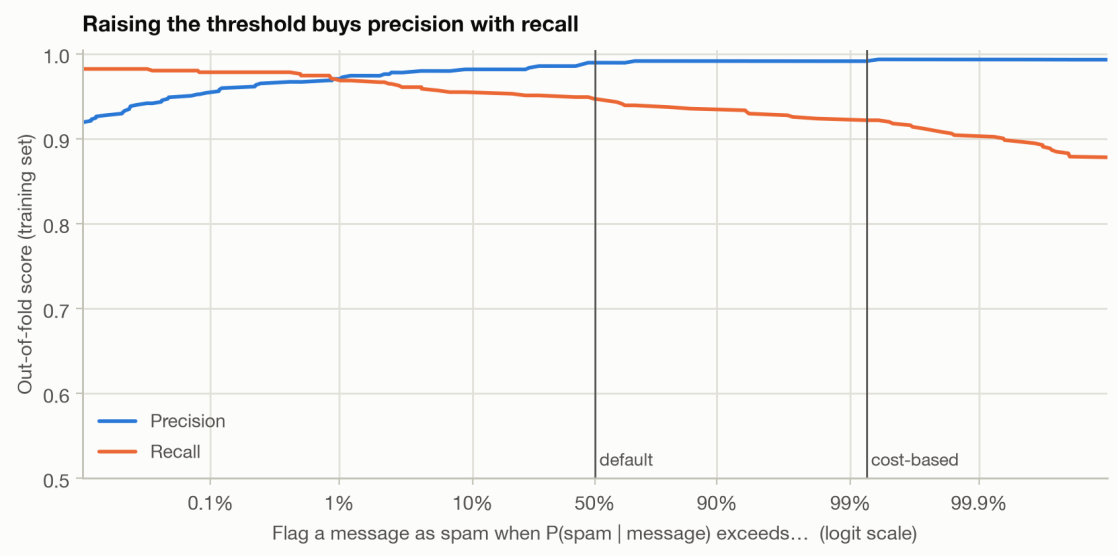


Figure 6. Precision and recall of the Bernoulli filter against the decision threshold (out-of-fold predictions on the training set, logit scale).

4.6 Error analysis

The final filter made 16 errors on the test set, all of them missed spam, and 12 of the 16 contain no phone number, money amount or link. They read like chat: jokes and trivia from subscription services, dating lines and a football score. For example:

- "Do you ever notice that when you're driving, anyone going slower than you is an idiot and everyone driving faster than you is a maniac?" (P(spam) = 4×10^{-14})
- "In The Simpsons Movie released in July 2007 name the band that died at the start of the film? A-Green Day, B-Blue Day, C-Red Day. (Send A, B or C)"
- "Latest News! Police station toilet stolen, cops have nothing to go on!"

Word evidence alone cannot separate these from ham. The two nearest misses (P(spam) ≈ 0.05) are a one-line teaser with a web link and "ringtoneking 84484", which carries only a short code.

4.7 How much training data Naive Bayes needs

With 66 training messages (about 8 of them spam), the three Naive Bayes models reach F1 of 0.77–0.84 while logistic regression reaches 0.67 (Figure 7). Logistic regression closes most of the gap by about 1,000 messages but never overtakes the Naive Bayes word models, the pattern Ng and Jordan (2002) described for generative versus discriminative classifiers. Gaussian Naive Bayes on the statistics plateaus near 0.89, because eight numbers per message cannot carry as much information as the words.

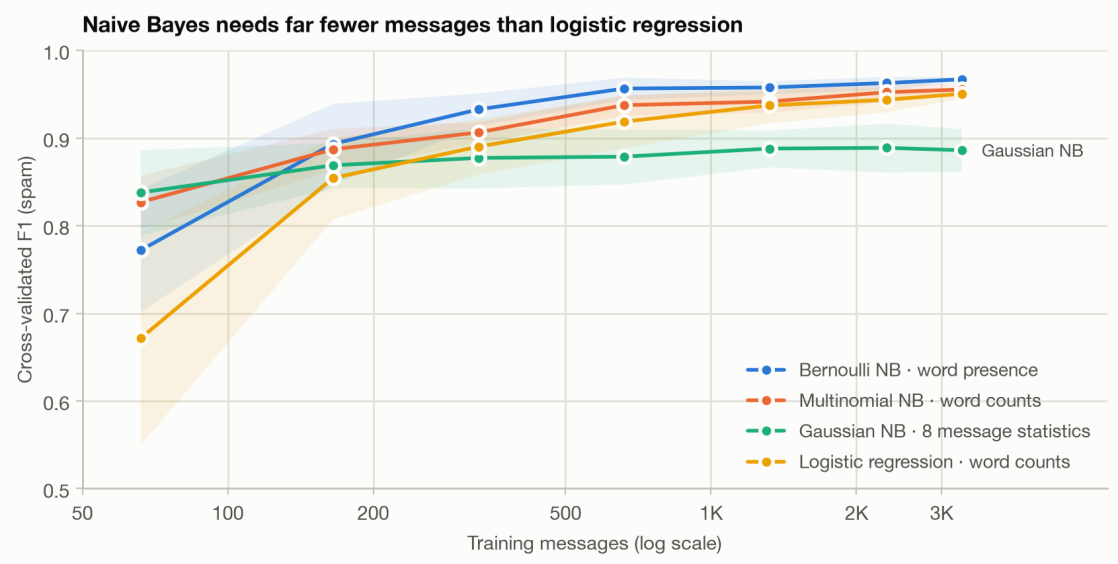


Figure 7. Cross-validated F1 against training-set size (5 folds; bands show ±1 standard deviation).

4.8 Checking the assumptions

Assumption	What the data show	Consequence
Features are Gaussian (Gaussian NB)	log(1 + x) makes length almost symmetric (skewness 2.4 → 0.0), but counts stay skewed (links 8.7 → 7.9)	Gaussian NB on statistics flags long, number-heavy ham
Features are independent given the class	characters and words correlate at 0.97 within ham	the hybrid counts length twice and adds false alarms
Posterior probabilities are meaningful	99% of Bernoulli and 89% of multinomial test scores claim more than 99.9% certainty	scores rank messages well but are not calibrated probabilities

Naive Bayes still classifies well despite these violations because a correct decision only needs the right class to score highest, not accurate probabilities (Domingos and Pazzani, 1997).

5. Discussion and Limitations

The results support the standard advice for Naive Bayes: choose the event model from the type of the features. Discrete word models (multinomial and Bernoulli) are clearly right for text; the Gaussian model is right for continuous measurements but wrong for sparse counts; and adding the two together is only as good as the independence between them. Bernoulli edged multinomial here because SMS messages are so short that counts carry little more than presence, which fits McCallum and Nigam's (1998) finding for small vocabularies.

Limitations:

1. **Collection artifacts.** One ham source hid its numbers behind <#>, so numbers, and phone numbers especially, look more spam-specific here than they would on a real phone. The placeholder was neutralised, but the hidden numbers cannot be restored.

2. **Age and place.** The messages were collected before 2011, with spam mostly from the UK and ham mostly from Singapore. Modern spam, such as parcel-delivery scams with short links, would need retraining.
3. **Over-confidence.** Most Naive Bayes scores claim more than 99.9% certainty, so they should be read as rankings, not risks.
4. **Small test set.** With 128 test spam messages, the F1 intervals are about ±0.03 wide, so differences smaller than that between models are not meaningful.
5. **One split.** Results come from a single 80/20 split; the cross-validation ranking on the training set agrees with the test ranking for the clearly separated models.

6. Conclusion

- **RQ1.** Naive Bayes is an effective SMS spam filter: the final Bernoulli model classified 98.4% of unseen messages correctly, caught 87.5% of spam and flagged no legitimate message.
- **RQ2.** Use multinomial or Bernoulli Naive Bayes for words and Gaussian Naive Bayes for continuous statistics. Gaussian on word counts fails (recall 0.68), and combining the two only helps when their features are independent; here, correlated length features added false alarms.
- **RQ3.** Phone numbers and reward words give spam away; what still gets through is spam written like chat.

Future work: retrain on recent messages, add character n-grams to catch obfuscated words such as “fr33”, calibrate the probabilities (Platt or isotonic scaling) before choosing a threshold, and drop redundant statistics before combining them with the words.

References

Almeida, T. A., Gómez Hidalgo, J. M., and Yamakami, A. (2011). Contributions to the study of SMS spam filtering: New collection and results. In *Proceedings of the 11th ACM Symposium on Document Engineering* (pp. 259–262). https://doi.org/10.1145/2034691.2034742

Domingos, P., and Pazzani, M. (1997). On the optimality of the simple Bayesian classifier under zero-one loss. *Machine Learning, 29*(2–3), 103–130. https://doi.org/10.1023/A:1007413511361

McCallum, A., and Nigam, K. (1998). A comparison of event models for naive Bayes text classification. In *AAAI-98 Workshop on Learning for Text Categorization*. https://openalex.org/W1550206324

Metsis, V., Androutsopoulos, I., and Paliouras, G. (2006). Spam filtering with naive Bayes: Which naive Bayes? In *Third Conference on Email and Anti-Spam (CEAS 2006)*. https://openalex.org/W2160536005

Ng, A. Y., and Jordan, M. I. (2002). On discriminative vs. generative classifiers: A comparison of logistic regression and naive Bayes. In *Advances in Neural Information Processing Systems 14*. https://openalex.org/W2163614729

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Sahami, M., Dumais, S., Heckerman, D., and Horvitz, E. (1998). A Bayesian approach to filtering junk e-mail. In *AAAI-98 Workshop on Learning for Text Categorization*. https://openalex.org/W1648885110

Appendix A. Reproducibility

Item	Detail
Notebook	`sms_spam_naive_bayes.ipynb`: 58 cells, saved with outputs
Environment	Python 3.12 · numpy 2.5.4 · pandas 3.0.6 · scikit-learn 1.9.1 · scipy 1.18.1 · matplotlib 3.11.2 (`requirements.txt`)
Run	`pip install -r requirements.txt`, then `jupyter nbconvert --to notebook --execute --inplace sms_spam_naive_bayes.ipynb`
Runtime	about 30 seconds on a laptop; random seed 5830 throughout
Outputs	`figures/` (8 PNG figures), `results/` (CSV and JSON tables used by the report and slides)
Slides	`slides/sms_spam_naive_bayes.pptx`: 8 slides with speaker notes, charts built from `results/`
Data	`data/sms_spam_collection/SMSSpamCollection`, downloaded from the UCI repository by the notebook if missing